

Resumen de mejores practicas de C++

Basado en las C++ Core Guidelines de Bjarne Stroustrup y Herb Sutter.

Que resume este apunte Una seleccion practica de recomendaciones para escribir C++ moderno, seguro, mantenible y con buen rendimiento sin caer en tecnicas viejas o propensas a errores.	Fuente base C++ Core Guidelines, documento vivo publicado en isocpp.github.io , actualizado el 8 de julio de 2025.
---	--

Idea central de las guidelines: usar C++ moderno para lograr seguridad de tipos, manejo correcto de recursos, interfaces claras y codigo simple que siga siendo eficiente.

En la practica, esto se traduce en preferir RAII, ``const``, inicializacion clara, smart pointers cuando hay ownership, ``std::vector``/``std::array`` frente a arreglos C, y funciones y clases con responsabilidades bien definidas.

Checklist rapida

Tema	Regla rapida
Ownership	Si un objeto posee un recurso, modelalo explicitamente con RAII y ownership claro.
Memoria	Evita <code>`new`</code> / <code>`delete`</code> manual; preferi <code>`make_unique`</code> , <code>`make_shared`</code> , objetos locales y contenedores.
Interfaces	Un <code>`T*`</code> o <code>`T&`</code> suele ser no-owning; si no puede ser nulo, expresalo con <code>`not_null`</code> .
Secuencias	No pases arreglos como puntero solo; usa <code>`span`</code> , <code>`vector`</code> o <code>`array`</code> .
Funciones	Una funcion = una operacion logica; mantenelas cortas y con intencion visible.
Tipos	Usa <code>`class`</code> si hay invariantes; <code>`struct`</code> si los miembros pueden variar independientemente.
Inicializacion	Preferi <code>`{}`</code> y declara variables solo cuando ya tengas con que inicializarlas.
Inmutabilidad	Por defecto, usa <code>`const`</code> ; usa <code>`constexpr`</code> cuando el valor pueda resolverse en compilacion.
Errores	Usa excepciones para fallas reales y diseña la estrategia de errores desde el inicio.
Contenedores	Por defecto, usa <code>`vector`</code> ; evita arreglos C y desbordes de rango.
Estilo peligroso	Evita macros, casts estilo C, <code>`goto`</code> , y codigo que oculte ownership o tamanos.

1. Filosofia general

Escribe la idea directamente en el código. Las guidelines priorizan expresar intención, usar C++ estándar, buscar seguridad de tipos y hacer todo lo posible en compilación antes que en tiempo de ejecución.

Busca seguridad sin sacrificar performance. La idea no es escribir C++ 'empobrecido', sino usar mejores abstracciones para reducir errores sin pagar sobrecostos innecesarios.

Prefiere datos inmutables. Menos mutabilidad suele significar menos estados sorpresivos y menos bugs.

2. Recursos y ownership

Usa RAII para cualquier recurso. No solo memoria: también archivos, locks, sockets, handles, etc. El recurso debe quedar atado a la vida útil del objeto que lo administra.

Evita `new` y `delete` explícitos. Si necesitas asignación dinámica, entrega el resultado inmediatamente a un objeto gestor.

Ownership explícito. `unique_ptr` representa ownership único. `shared_ptr`, ownership compartido. Si no posees el objeto, un puntero o referencia crudos son observadores, no dueños.

3. Interfaces mas seguras

No transfieras ownership con `T*` o `T&`. Las guidelines recomiendan que el ownership nunca quede ambiguo.

Si un puntero no puede ser nulo, dilo. Modelalo con `not_null` para que la restricción sea visible y verificable.

No pases arreglos como puntero sin tamaño. Usa `span` para preservar tamaño y reducir errores de rango.

En uso general, recibe `T*` o `T&`, no smart pointers. Pide smart pointers como parámetro solo si realmente necesitas expresar semántica de vida útil.

4. Funciones

Una función = una operación lógica. Eso mejora comprensión, testing y reutilización.

Mantenerlas cortas y simples. Menos caminos, menos estados y menos efectos colaterales.

Usa `constexpr` cuando corresponda. Si algo puede evaluarse en compilación, mejorarlo ahí ayuda a rendimiento y chequeos.

Usa `noexcept` cuando la función no deba fallar. La firma comunica mejor la semántica y ayuda a optimizaciones.

Prefiere funciones puras cuando puedas. Menos dependencias externas, más predictibilidad.

5. Clases y tipos

`class` si hay invariantes, `struct` si no. La convención ayuda a leer que objetos tienen reglas internas que deben mantenerse.

Regla de cero. Si puedes evitar definir constructor de copia, move, asignaciones y destructor, hazlo. Deja que los miembros manejen sus propios recursos.

Si defines una operación especial, revisa todas. Si tocas copia/move/destructor, debes considerar el conjunto completo para mantener coherencia.

6. Inicializacion, const y enums

Prefiere inicializacion con `{}`. Es mas uniforme y evita varios casos de narrowing o inicializacion confusa.

Declara una variable solo cuando ya tengas con que inicializarla. Reduce el riesgo de usar valores basura o de dar usos multiples a una misma variable.

Por defecto, usa `const`. Haz objetos y metodos `const` salvo que de verdad deban mutar estado.

Usa `constexpr` para valores calculables en compilacion. Es una forma fuerte de documentar y verificar intencion.

Prefiere `enum class` frente a `enum` plano. Evita conversiones implicitas sorprendivas a `int`.

7. Expresiones, iteracion y estilo

Prefiere `range-for` cuando puedas. Suele ser mas legible, menos propenso a errores y suficientemente eficiente.

Evita macros para manipular texto de programa. Las macros rompen reglas de alcance y tipos, y complican herramientas.

Si necesitas cast, usa casts con nombre. Evita casts estilo C; hacen mas dificil entender que conversion se esta realizando.

Evita `goto` y codigo de flujo innecesariamente enredado. Suele esconder errores y dificulta mantenimiento.

8. Contenedores y biblioteca estandar

Prefiere `std::vector` por defecto. Las guidelines lo recomiendan como contenedor general por seguridad, locality y rendimiento.

Usa `std::array` o `vector` en lugar de arreglos C. Los arreglos C pierden informacion de tamano y empeoran seguridad.

Evita errores de rango. El acceso fuera de limites es una fuente clasica de bugs graves y problemas de seguridad.

No uses `memcpy` / `memset` sobre tipos no trivially-copyable. Puedes romper la semantica del objeto.

9. Errores y robustez

Define la estrategia de errores temprano. Es muy dificil injertarla correctamente al final del proyecto.

Usa excepciones para indicar que una funcion no puede cumplir su tarea. No para control de flujo ordinario.

Deja que los constructores establezcan invariantes o lancen. Un objeto medio construido es una fuente de problemas.

RAII tambien simplifica el manejo de errores. Si hay excepciones, los destructores limpian automaticamente.

Ejemplos cortos

1) Ownership y RAII

```
std::unique_ptr<Foo> f = std::make_unique<Foo>();

// mejor que:
Foo* f = new Foo();
// ...
delete f;
```

2) Secuencias: mejor `span` que puntero + tamaño

```
void procesar(std::span<int> xs) {
    for (int& x : xs) { /* ... */ }
}
```

3) `const` y range-for

```
for (const auto& x : v) {
    std::cout << x << '\n';
}
```

4) `enum class`

```
enum class Estado { pendiente, ok, error };

Estado e = Estado::ok;
```

Antipatrones que conviene evitar

- Punteros crudos con ownership ambiguo.
- ``new`` y ``delete`` esparcidos por el código.
- Interfaces del tipo ``(T*, int)`` o ``int[]`` para representar secuencias.
- Arreglos C cuando un ``vector`` o ``array`` resolvería mejor el problema.
- Macros para constantes o pseudo-funciones.
- Casts estilo C.
- Variables no inicializadas o reutilizadas para propósitos distintos.
- Funciones demasiado largas con muchas responsabilidades.
- Mutabilidad innecesaria.
- Uso indiscriminado de ``shared_ptr`` cuando alcanza ``unique_ptr``.

Como estudiar esto sin abrumarte

- Primero internaliza 5 ideas: RAII, ownership claro, ``const``, inicialización con ``{}``, y ``vector/span`` en lugar de arreglos C.
- Después suma ``unique_ptr``, ``enum class``, ``range-for``, ``constexpr`` y la regla de cero.
- Recién al final profundiza en detalles más finos: contratos, perfiles, GSL, concurrencia y plantillas.

Fuente resumida

C++ Core Guidelines, documento vivo editado por Bjarne Stroustrup y Herb Sutter. Versión del sitio consultada: 8 de julio de 2025.

Este PDF no reemplaza la guía completa: funciona como apunte práctico para estudiar y programar con un criterio más moderno.